

Pertemuan 4

Debugging, Testing, Terminasi, dan Rekursi

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Rifky Fauzi

Triyana Muliawati, S.Si., M.Si.

Program Studi Matematika
Institut Teknologi Sumatera
Semester Ganjil 2026/2027

Cara belajar hari ini

- Mulai dari masalah: program salah, macet, atau hasilnya aneh.
- Tulis perhitungan manual untuk kasus kecil.
- Baru tulis algoritma dan kode Python.
- Terka kesalahan, perbaiki, lalu uji kembali.

Masalah 1: program jalan terus

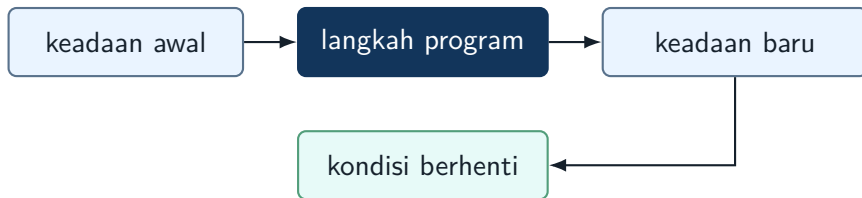
```
n = 5
while n > 0:
    print(n)
    n = n + 1
```

Diskusi cepat

- 1 Nilai pertama yang dicetak apa?
- 2 Setelah satu putaran, nilai n menjadi apa?
- 3 Apakah $n > 0$ makin dekat menjadi salah?

Program yang benar bukan hanya menghasilkan output. Program juga harus punya alasan mengapa prosesnya berhenti.

Loop: keadaan berubah menuju berhenti



Pertanyaan saat menulis loop

- Apa kondisi berhentinya?
- Variabel mana yang diubah setiap putaran?
- Ukuran apa yang makin kecil atau makin dekat ke batas?

Contoh ukuran yang turun

$$i = n, \quad i \leftarrow i // 2$$

Jika $n > 0$, nilai i akhirnya menjadi 0.

Hitung manual: tiga loop, tiga perilaku

Kode ringkas	Contoh lintasan	Halt?	Alasan
$i = n; i = i // 2$ selama $i > 0$	10, 5, 2, 1, 0	Ya	makin kecil menuju 0
$i = n; i = i - 2$ selama $i \neq 1$	7, 5, 3, 1	tergantung	ganjil positif berhenti; genap melewati 1
$i = n; i = i + 1$ selama $i > 0$	5, 6, 7, ...	Tidak	makin jauh dari 0

Latihan kelas

Untuk loop kedua, tulis lintasan mulai dari $n = 6$. Apa yang terjadi setelah $i = 2$?

Halt atau terminasi

Definisi kerja

Suatu program dikatakan *halt* untuk sebuah input jika program berhenti setelah jumlah langkah yang berhingga.

Loop

Kita mencari ukuran yang bergerak menuju kondisi berhenti.

$$i \rightarrow i - 1, \quad i \rightarrow i // 2, \quad |x_k^2 - n| \rightarrow 0.$$

Rekursi

Kita mencari submasalah yang lebih kecil dan base case.

$$n! = n(n - 1)!, \quad 0! = 1.$$

Masalah 2: program berhenti, tetapi hasilnya salah

```
def rata(data):  
    total = 0  
    for x in data:  
        total += x  
    return total / len(data)
```

Coba hitung manual

Data: [2, 4, 6, 8]

- Harusnya rata-rata:
 $(2 + 4 + 6 + 8)/4 = 5$.
- Kode mengembalikan nilai setelah membaca elemen pertama.
- Hasilnya: $2/4 = 0.5$.

Kesalahan ini tidak selalu menghasilkan error. Python jalan, tetapi logika program salah.

Memperbaiki: return setelah loop selesai

```
def rata(data):  
    if len(data) == 0:  
        return None  
    total = 0  
    for x in data:  
        total += x  
    return total / len(data)
```

Alur perhitungan

x	<i>total lama</i>	<i>total baru</i>
2	0	2
4	2	6
6	6	12
8	12	20

Akhirnya $20/4 = 5$.

Tiga jenis masalah program

Jenis	Gejala	Contoh
Syntax error	kode tidak dapat dibaca Python	titik dua hilang setelah <code>if</code>
Runtime error	kode mulai berjalan, lalu berhenti karena operasi tidak sah	pembagian dengan nol
Logical error	kode berjalan, tetapi hasil salah	penyebut rata-rata ditulis <code>len(data)+1</code>

Cara membaca error

Jangan mulai dari panik. Baca baris error, tipe error, lalu nilai variabel yang terlibat.

Contoh: mana jenis kesalahannya?

```
# A
if x > 0
    print(x)
```

```
# B
x = 3 / 0
```

```
# C
def rata(data):
    return sum(data) / (len(
        data)+1)
```

Tugas dua menit

Tentukan jenis kesalahan A, B, dan C. Untuk C, uji dengan data [2, 4, 6, 8].

Exception handling: fungsi rasio

```
def rasio(a, b):  
    try:  
        a = float(a)  
        b = float(b)  
        return a / b  
    except ValueError:  
        return "input bukan angka"  
    except TypeError:  
        return "tipe data tidak cocok"  
    except ZeroDivisionError:  
        return "pembagi nol"
```

Uji kasus

Input	Makna
('3.5', '2')	normal
('abc', 2)	gagal konversi
(3, 0)	pembagi nol
(None, 4)	tipe tidak cocok

Mengapa bukan except kosong?

Masalah dengan except:

except: menangkap terlalu banyak hal. Bug yang seharusnya terlihat bisa tersembunyi.

Lebih baik

Tangani jenis masalah yang kita pahami:

- `ValueError`: nilai tidak dapat dikonversi.
- `TypeError`: tipe data tidak sesuai.
- `ZeroDivisionError`: pembagian nol.

Prinsip

Jika error belum dipahami, biarkan terlihat saat pengembangan. Setelah jelas, baru tangani.

Debugging: dari gejala ke penyebab



Untuk kelas

Kalau mahasiswa bingung membaca kode, jangan langsung ke sintaks. Minta mereka mengisi tabel nilai variabel dari satu input kecil.

Masalah 3: akar kuadrat dengan Newton

Masalah matematika

Cari x sehingga

$$x^2 = n.$$

Definisikan $F(x) = x^2 - n$. Akar $F(x) = 0$ adalah $\sqrt{n}$.

Iterasi Newton

$$x_{k+1} = x_k - \frac{F(x_k)}{F'(x_k)} = \frac{1}{2} \left(x_k + \frac{n}{x_k} \right).$$

Syarat penting: $x_k \neq 0$.

Hitung manual Newton untuk $\sqrt{10}$

Ambil $x_0 = 5$. Rumus:

$$x_{k+1} = \frac{1}{2} \left(x_k + \frac{10}{x_k} \right).$$

k	x_k	$x_k^2 - 10$
0	5.0000	15.0000
1	3.5000	2.2500
2	3.1786	0.1033
3	3.1623	mendekati 0

Yang diamati

Galat $|x_k^2 - n|$ semakin kecil. Inilah alasan kita boleh berhenti setelah toleransi tercapai.

Kode Newton yang sengaja salah

```
def akar(n, tol=1e-10):  
    x = 0.0  
    while abs(x*x - n) > tol:  
        x = 0.5 * (x + n/x)  
    return x
```

Tebak masalahnya

- Apa yang terjadi saat n/x dan $x=0.0$?
- Bagaimana jika $n = 0$?
- Bagaimana jika $n < 0$?
- Bagaimana kalau iterasi tidak membaik?

Newton yang lebih aman

```
def akar(n, tol=1e-10, max_iter=100):  
    if n < 0:  
        return None  
    if n == 0:  
        return 0.0  
    x = n / 2 if n >= 1 else 1.0  
    for k in range(max_iter):  
        if abs(x*x - n) <= tol:  
            return x  
        x = 0.5 * (x + n/x)  
    return None
```

Makna desain

Ada penanganan domain, tebakan awal tidak nol, toleransi, dan batas iterasi.

Masalah 4: fungsi prima dan kondisi batas

Definisi

Bilangan bulat positif n disebut prima jika $n > 1$ dan hanya punya pembagi positif 1 dan n .

Mengapa cukup sampai $\sqrt{n}$?

Jika $n = ab$ dan $a > \sqrt{n}$, maka $b < \sqrt{n}$.
Jadi salah satu faktor nontrivial pasti tidak melebihi $\sqrt{n}$.

Contoh manual

Untuk $n = 49$, $\sqrt{49} = 7$. Jika percobaan pembagi berhenti sebelum 7, faktor 7 tidak pernah diuji.

Kode prima yang salah

```
def is_prima(n):  
    if n <= 1:  
        return False  
    d = 2  
    while d*d < n: # salah di batas  
        if n % d == 0:  
            return False  
        d += 1  
    return True
```

Cari pembongkar bug

Uji:

$$n = 4, 9, 25, 49.$$

Apa yang terjadi ketika $d^2 = n$?

Perbaiki: gunakan `while d*d <= n`. Kondisi batas sering lebih penting daripada kasus besar.

Testing: bukan sekadar “kode jalan”

Ide

Testing berarti memilih kasus yang jawabannya sudah diketahui atau sifatnya sudah pasti.

Kasus normal

- input sederhana
- hasil mudah dihitung manual

Kasus batas

- data kosong
- pembagi nol
- $n = 0, 1, 2$

Sifat matematis

- simetri jarak: $d(x, y) = d(y, x)$
- nonnegatif
- nilai khusus yang diketahui

Jarak Euclid: dari rumus ke algoritma

Untuk dua vektor

$$x = (x_1, \dots, x_m), \quad y = (y_1, \dots, y_m),$$

jarak Euclid didefinisikan sebagai

$$d(x, y) = \sqrt{\sum_{i=1}^m (x_i - y_i)^2}.$$

Algoritma manual

- 1 Pastikan panjang x dan y sama.
- 2 Untuk tiap indeks, hitung selisih $x_i - y_i$.
- 3 Kuadratkan selisih, jumlahkan, lalu ambil akar.

Implementasi jarak dan assert

```
from math import sqrt

def jarak(x, y):
    assert len(x) == len(y)
    total = 0
    for i in range(len(x)):
        total += (x[i] - y[i])**2
    return sqrt(total)
```

Tes yang masuk akal

```
assert jarak([0,0],[3,4]) == 5
assert jarak([1,2],[1,2]) == 0
assert jarak([-1,2],[2,6]) == 5
assert jarak([1,2],[3,4]) == jarak
        ([3,4],[1,2])
```

Rekursi dari masalah matematika

Pertanyaan matematika

Bagaimana menghitung $5!$?

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1.$$

Pola yang terlihat

$$5! = 5 \cdot 4!, \quad 4! = 4 \cdot 3!, \quad 3! = 3 \cdot 2!.$$

Definisi rekursif

$$n! = n(n-1)!, \quad 0! = 1.$$

Base case: $0!$. Recursive case: $n(n-1)!$.

Faktorial: rekursif dan iteratif

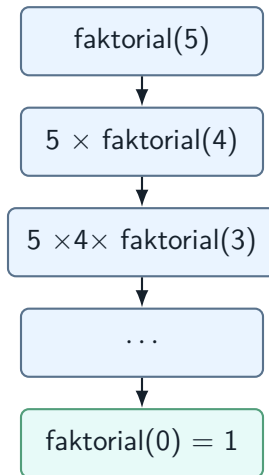
```
def faktorial(n):  
    if n == 0:  
        return 1  
    return n * faktorial(n-1)
```

```
def faktorial_iter(n):  
    hasil = 1  
    for k in range(1, n+1):  
        hasil *= k  
    return hasil
```

Diskusi

Keduanya menghitung nilai yang sama. Versi rekursif menyimpan pemanggilan yang belum selesai; versi iteratif menyimpan akumulasi hasil.

Call stack untuk 5!



Yang harus dicek

Apakah setiap pemanggilan bergerak menuju $n = 0$? Jika tidak, rekursi tidak berhenti.

Barisan rekursif: hitung dulu, kode kemudian

Diberikan

$$a_0 = 2, \quad a_n = 3a_{n-1} - 2,$$

untuk $n \geq 1$.

n	Rumus	Nilai
0	diberikan	2
1	$3a_0 - 2$	4
2	$3a_1 - 2$	10
3	$3a_2 - 2$	28

Sebelum menulis kode

Tentukan base case, recursive case, dan contoh nilai kecil untuk memeriksa program.

Kode rekursif barisan

```
def a(n):  
    if n == 0:  
        return 2  
    return 3*a(n-1) - 2
```

Tes cepat

```
assert a(0) == 2  
assert a(1) == 4  
assert a(2) == 10  
assert a(3) == 28
```

Rumus eksplisit

Dengan mengamati nilai: 2, 4, 10, 28, ..., mahasiswa dapat menebak pola, lalu memverifikasi dengan induksi atau pengujian numerik.

Mencari rumus eksplisit secara bertahap

Untuk

$$a_n = 3a_{n-1} - 2,$$

carilah titik tetap $Li = 3L - 2$, sehingga

$$L = 1.$$

Definisikan $b_n = a_n - 1$. Maka

$$b_n = a_n - 1 = 3a_{n-1} - 3 = 3(a_{n-1} - 1) = 3b_{n-1}.$$

Karena $b_0 = a_0 - 1 = 1$, maka

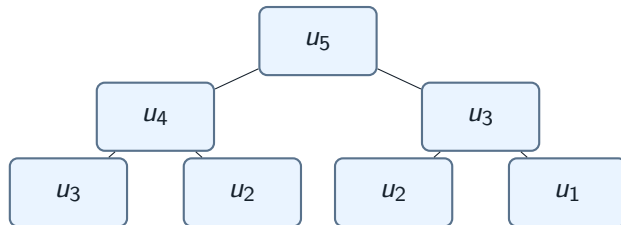
$$b_n = 3^n, \quad a_n = 3^n + 1.$$

Rekursi yang menghitung berulang

Diberikan

$$u_0 = 1, \quad u_1 = 3, \quad u_n = u_{n-1} + 2u_{n-2}.$$

Untuk u_5 :



Amati

Submasalah u_3 dan u_2 muncul lebih dari sekali. Ini menjelaskan mengapa rekursi langsung dapat mahal.

Kode rekursif langsung: benar, tetapi bisa mahal

```
def u(n):  
    if n == 0:  
        return 1  
    if n == 1:  
        return 3  
    return u(n-1) + 2*u(n-2)
```

Untuk u_5

$$u_2 = 3 + 2(1) = 5,$$

$$u_3 = 5 + 2(3) = 11,$$

$$u_4 = 11 + 2(5) = 21,$$

$$u_5 = 21 + 2(11) = 43.$$

Diskusi

Kalau nilai kecil dapat dihitung manual dengan tabel, mengapa rekursi langsung tetap melakukan banyak pemanggilan ulang?

Dari recursive case ke tabel iteratif

Definisi

$$u_n = u_{n-1} + 2u_{n-2}.$$

Untuk menghitung u_n , sebenarnya cukup menyimpan dua nilai terakhir.

n	u_{n-2}	u_{n-1}	u_n
2	1	3	5
3	3	5	11
4	5	11	21
5	11	21	43

Pesan

Rekursi membantu mendeskripsikan masalah. Implementasi tidak selalu harus rekursif jika tabel iteratif lebih jelas.

Membaca kode salah: prosedur kelas

- 1 Tulis satu input kecil yang jawabannya dapat dihitung manual.
- 2 Buat tabel nilai variabel per langkah.
- 3 Tandai baris pertama yang membuat nilai keluar dari harapan.
- 4 Perbaiki satu hal saja.
- 5 Uji ulang dengan kasus normal dan kasus batas.

Aturan diskusi

Kalau maju ke papan, tidak harus langsung benar. Yang dicari adalah alasan mengapa program salah.

Latihan kelas: diagnosa cepat

```
def jumlah_positif(data):  
    total = 0  
    for x in data:  
        if x > 0:  
            total += x  
        else:  
            return total  
    return total
```

Pertanyaan

Untuk data = [2, 3, -1, 4, 5], apakah fungsi memang menjumlahkan semua bilangan positif? Jika tidak, baris mana yang menyebabkan masalah?

Latihan kelas: buat test case pembongkar bug

```
def maksimum(data):  
    m = 0  
    for x in data:  
        if x > m:  
            m = x  
    return m
```

Pertanyaan

Fungsi ini tampak benar untuk [1, 4, 2]. Cari satu data yang membuat fungsi salah. Mengapa?

Petunjuk: coba data yang seluruh elemennya negatif.

Untuk loop dan rekursi

- Tulis kondisi berhenti.
- Tulis ukuran yang bergerak menuju berhenti.
- Uji dengan kasus kecil.

Untuk debugging

- Hitung manual dulu.
- Telusuri nilai variabel.
- Cari baris pertama yang menyimpang.

Untuk testing

- Uji kasus normal.
- Uji kondisi batas.
- Uji sifat matematis.
- Gunakan assert untuk cek cepat.

Yang dikumpulkan

- 1 Fungsi `rasio(a,b)` dengan penanganan `ValueError`, `TypeError`, dan `ZeroDivisionError` secara terpisah.
- 2 Perbaiki fungsi rata-rata dan penanganan data kosong.
- 3 Implementasi Newton yang aman untuk $n = 0$, $n > 0$, dan $n < 0$.
- 4 Fungsi `jarak(x,y)` dengan `assert` dan minimal empat test case.
- 5 Satu fungsi rekursif barisan beserta hitung manual tiga nilai awal.

Materi quiz

- Membedakan syntax error, runtime error, dan logical error.
- Menentukan output kode dengan `return` terlalu dini.
- Membaca loop dan menentukan apakah halt.
- Menentukan base case dan recursive case.
- Mengisi satu langkah iterasi Newton.
- Menentukan test case yang membongkar bug.

Cara belajar

Prediksi dulu di kertas. Jalankan Python hanya setelah alasan manual ditulis.



Kalimat kunci

Jika mahasiswa belum bisa menghitung satu contoh kecil dengan tangan, kode Python akan terasa seperti menebak. Mulai dari nilai kecil, tabel, dan alasan berhenti.